Generative Adversial Network

GAN

Dr. Pravin Kumar

School of Computer Science, UPES Dehradun

Limitations of Variational Autoencoders

1. Gaussian Assumption

- Approximate posterior assumed Gaussian:

$$q_\phi(z \mid x) = \mathcal{N}\left(\mu_\phi(x), \sigma^2_\phi(x)\right)$$

- Prior fixed as:

$$p(z) = \mathcal{N}(0, I)$$

- Eases training, but real data structure is often more complex

2. Blurry Generated Images

- Pixel-wise loss (e.g. MSE) compares generated vs. original image
- Suppose $x^{(1)} \xrightarrow{\text{Encoder}} z^{(1)}$ and $x^{(2)} \xrightarrow{\text{Encoder}} z^{(2)}$, with $z^{(1)} \approx z^{(2)} \approx z$
- Decoder receives nearly the same z for both $\Rightarrow$ averages uncertain outputs
- Result: sharp edges and fine detail get washed out

Trade-off Between Reconstruction and KL Divergence

The VAE loss has two parts:

$$\mathcal{L}_{VAE} = \mathcal{L}_{reconstruction} + D_{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$$

These two terms try to achieve different goals.

Trade-off Between Reconstruction and KL Divergence

Reconstruction Loss

- Wants the decoder to reconstruct the input accurately:

$$x \approx \hat{x}$$

- Therefore, the encoder tries to store **more** information about the input in z

Trade-off Between Reconstruction and KL Divergence

KL-Divergence Loss

- Wants the latent distribution to stay close to the standard normal prior:

$$q_\phi(z \mid x) \approx \mathcal{N}(0, I)$$

- Therefore, it prevents the encoder from storing **too much** input-specific information

Motivation for GANs

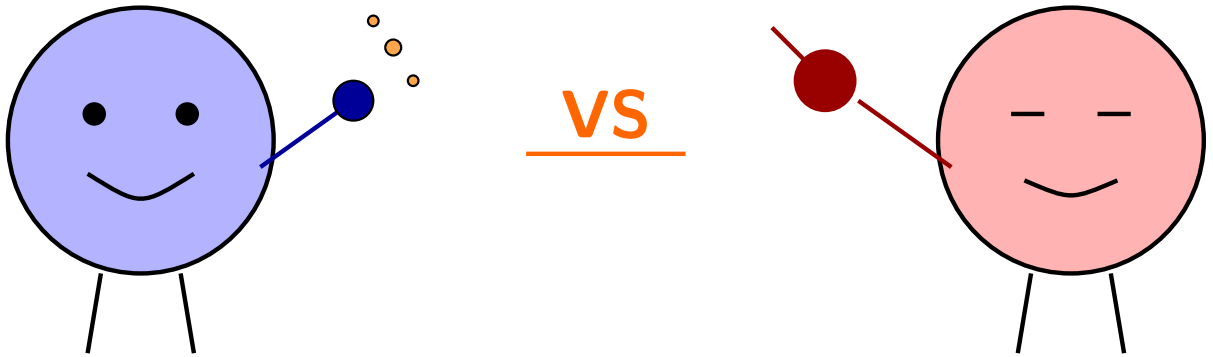
The main motivation for GANs is:

To generate highly realistic data without explicitly defining a probability distribution or reconstruction loss?

Fundamental Idea of Adversarial Learning

Competing with each other

Two neural networks are needed



Generator (G)

tries to *create* fake data

Discriminator (D)

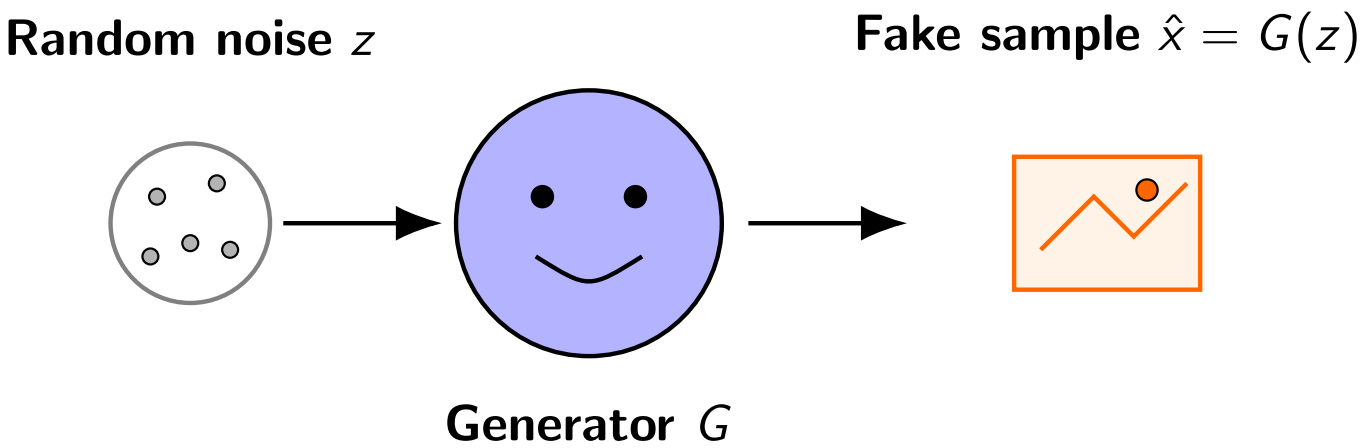
tries to *detect* fake data

These two neural networks learn by competing with each other.

Generator G

The generator generates fake samples from random noise:

$$z \longrightarrow G(z) = \hat{x}$$

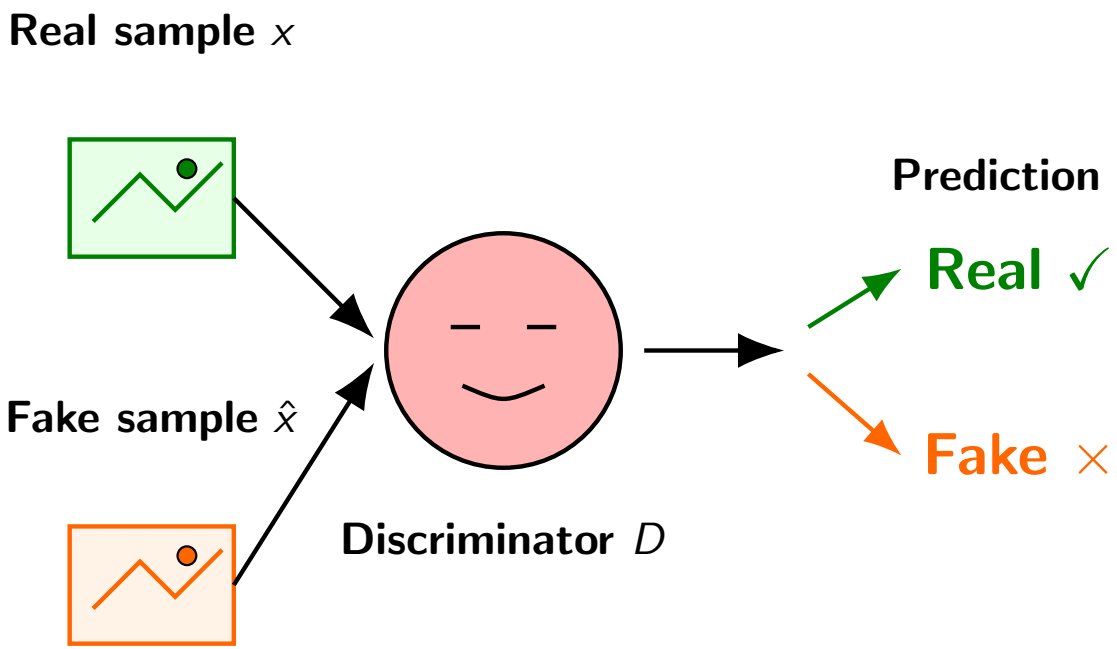


Its goal is: Generate samples that look real.

Discriminator D

The discriminator receives both real and generated samples and predicts whether each sample is real or fake:

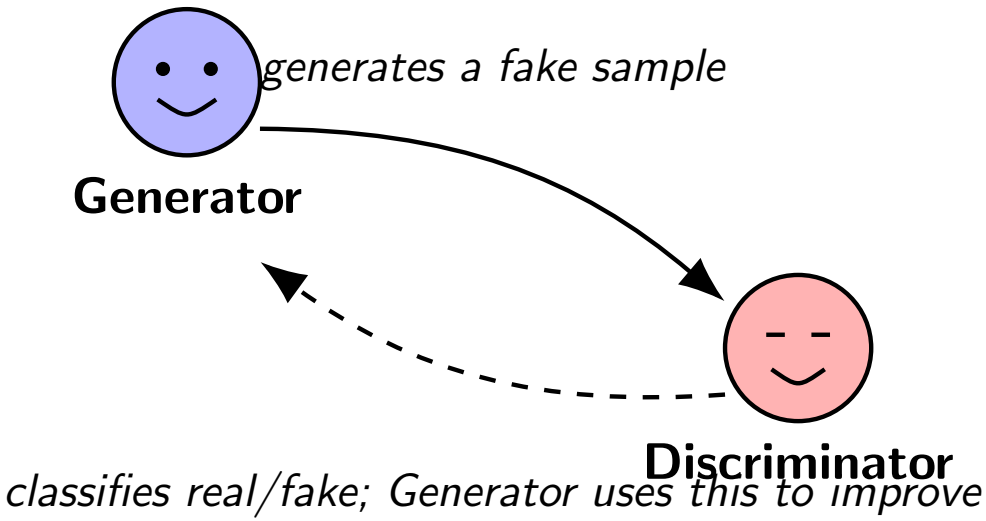
$$D(x) \longrightarrow \text{real or fake}$$



Its goal is: Correctly distinguish real samples from generated samples.

How the Competition Works

Discriminator learns to tell real samples from fake ones. **Generator** learns to fool the discriminator.



(the cycle repeats)

VAE vs. GAN: Which Way Does the Information Flow?

VAE: starts from data x

- Encoder **derives** z from a real input:

$$x \xrightarrow{\text{Encoder}} z$$

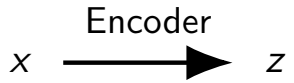
- z depends on x — it is a compressed representation of *that specific* input

GAN: starts from noise z

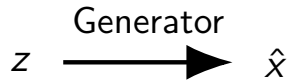
- No encoder. z is sampled from a fixed prior:

$$z \sim \mathcal{N}(0,1) \xrightarrow{\text{Generator}} \hat{x}$$

- z does **not** depend on x — the generator never looks at real data directly

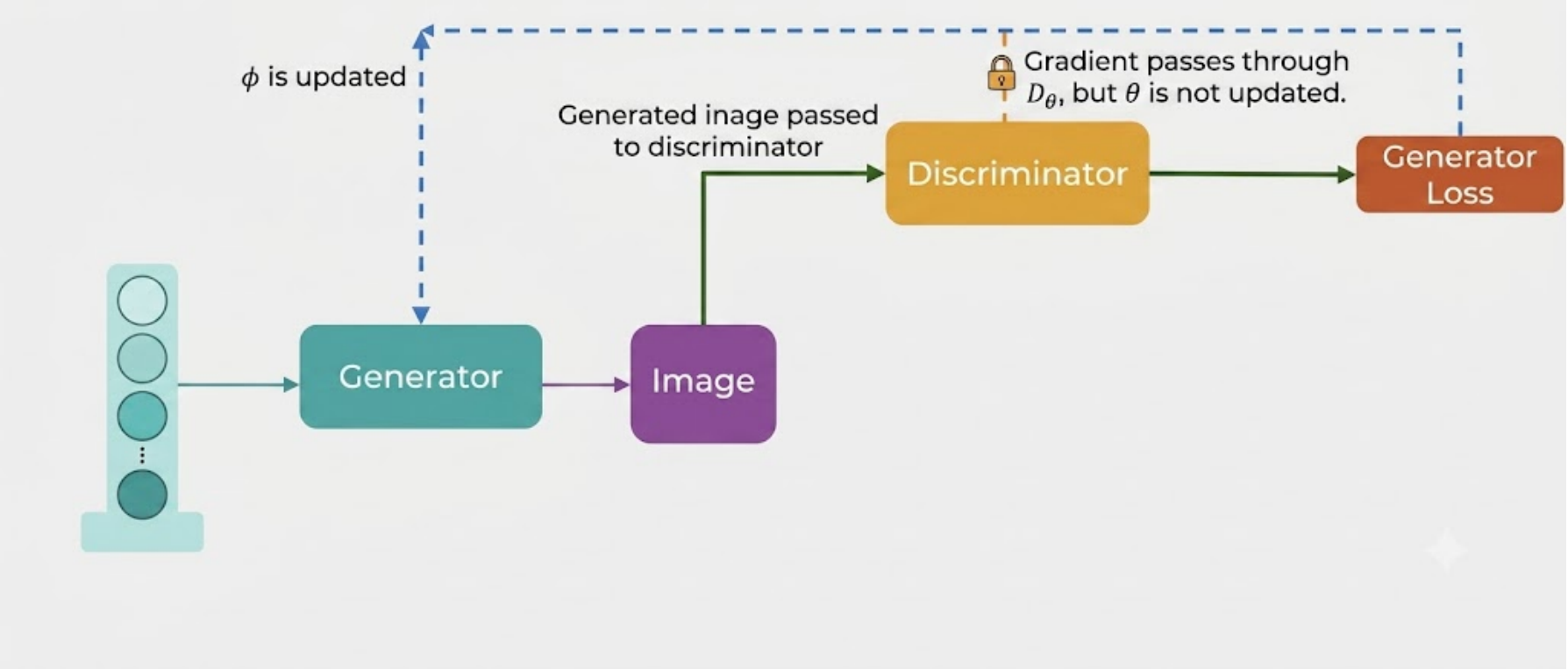


z depends on x



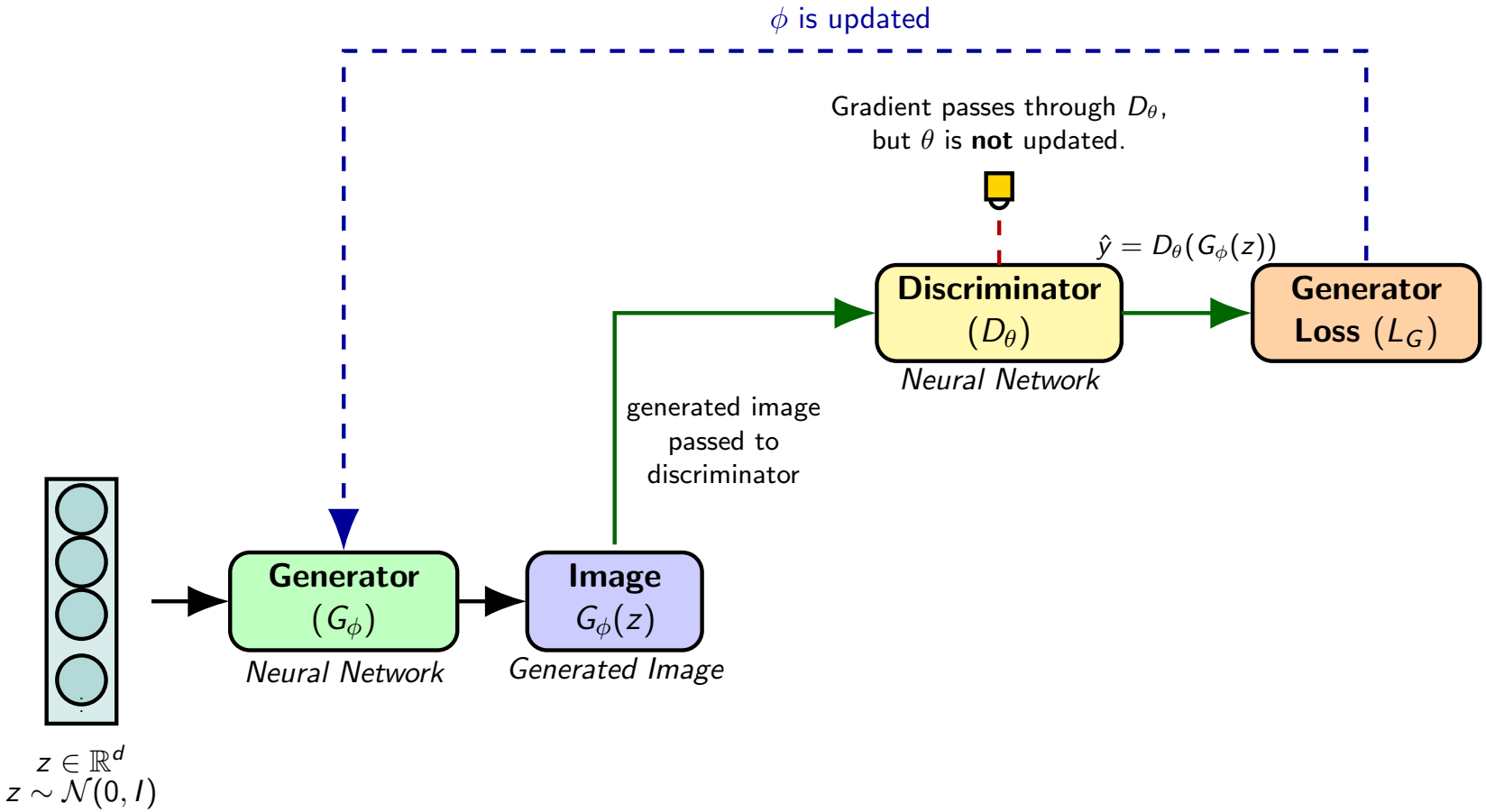
z is independent of x

Generator Training Pass

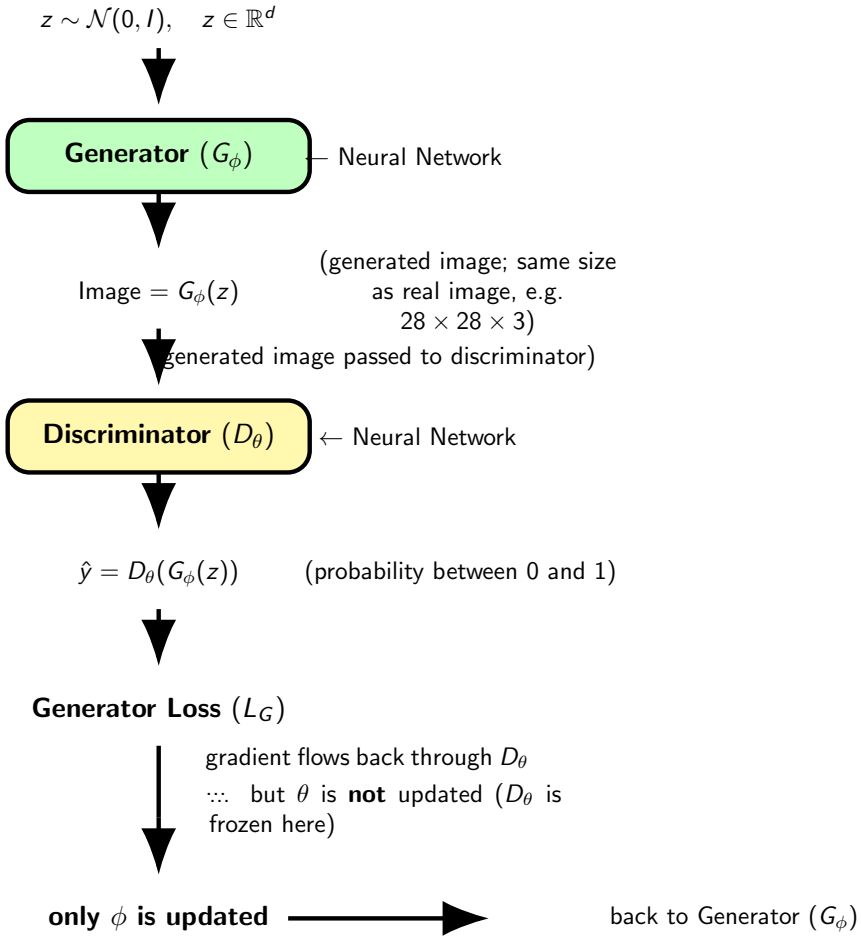


- Generator produces an image from noise z
- Image is passed to the Discriminator D_θ
- Gradient flows back through D_θ, but only ϕ (Generator's weights) is updated — θ stays frozen

Generator Training Pass



Generator Training Pass



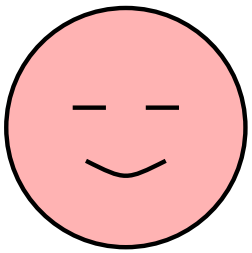
Why Is θ Not Updated Here?

"I'll score your fake image and tell you how unconvincing it is. . .

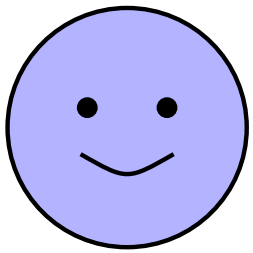
. . . but I **won't** change my own judgment based on this round."

"Got it — I'll use your feedback to improve *myself*.

Your weights θ stay frozen; only my weights ϕ update."



Discriminator (D_θ)



Generator (G_ϕ)

The discriminator acts only as a *scoring function* here — like an art critic whose standards don't shift just because they looked at one forgery.

Generator Input: The Noise Vector z

The generator starts from a random noise vector:

$$z \sim \mathcal{N}(0, I), \qquad z \in \mathbb{R}^2$$

To trace the GAN by hand, we fix a single sample of z:

$$z = \begin{bmatrix} 0.50 \\ -0.30 \end{bmatrix}$$

Real data (a single 2×2 image the generator will try to imitate):

$$x = \begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix} \in \mathbb{R}^{2\times 2}$$

Generator Forward Pass: Classwork

The generator is a single linear layer:

$$G_\phi(z) = Wz + b$$

Given:

$$W = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 1 & -1 \end{bmatrix}, \qquad b = \begin{bmatrix} 0 \\ 0 \\ 0.5 \\ -0.5 \end{bmatrix}, \qquad z = \begin{bmatrix} 0.50 \\ -0.30 \end{bmatrix}$$

Question: Compute $\hat{x} = G_\phi(z)$ by hand, then reshape it into a 2×2 image.

Generator Forward Pass: Worked Solution

Row-by-row computation: $\hat{x}_i = W_i \cdot z + b_i$

$$\begin{aligned}
\hat{x}_1 &= (1)(0.50) + (0)(-0.30) + 0 &&= 0.50 \\
\hat{x}_2 &= (0)(0.50) + (1)(-0.30) + 0 &&= -0.30 \\
\hat{x}_3 &= (1)(0.50) + (1)(-0.30) + 0.5 &&= 0.70 \\
\hat{x}_4 &= (1)(0.50) + (-1)(-0.30) - 0.5 &&= 0.30
\end{aligned}$$

Reshaped into a 2×2 fake image:

$$\hat{x} = G_\phi(z) = \begin{bmatrix} 0.50 & -0.30 \\ 0.70 & 0.30 \end{bmatrix}$$

Compare to the real image $x = \begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}$ — the generator starts out far from real; training is what closes this gap.

Discriminator Forward Pass: Classwork

The discriminator is a linear layer followed by a sigmoid:

$$D_\theta(x) = \sigma\left(w_d \cdot \text{flatten}(x) + b_d\right)$$

Given:

$$w_d = \begin{bmatrix} 0.1 & 0.1 & 0.1 & 0.1 \end{bmatrix}, \qquad b_d = -1$$

$$x = \begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}, \qquad \hat{x} = \begin{bmatrix} 0.50 & -0.30 \\ 0.70 & 0.30 \end{bmatrix}$$

Question: Compute $D_\theta(x)$ and $D_\theta(\hat{x})$ by hand.

Discriminator Forward Pass: Worked Solution

On the real image $x = [2, 4, 6, 8]$:

$$\text{logit} = 0.1(2 + 4 + 6 + 8) - 1 = 0.1(20) - 1 = 1.0$$
$$D_\theta(x) = \sigma(1.0) \approx 0.731$$

On the fake image $\hat{x} = [0.50, -0.30, 0.70, 0.30]$:

$$\text{logit} = 0.1(0.50 - 0.30 + 0.70 + 0.30) - 1 = 0.1(1.2) - 1 = -0.88$$
$$D_\theta(\hat{x}) = \sigma(-0.88) \approx 0.293$$

The discriminator leans toward "real" for x (0.731) and toward "fake" for $\hat{x}$ (0.293) — even with untrained weights, this gives us numbers to build the loss from.

Generator Loss

Consider BCE Loss:

$$L_{BCE} = -\left[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\right]$$

During generator training, the generator wants the discriminator's prediction for the generated image to become close to 1:

Generator's goal: make the discriminator classify the generated image as real

$$y = \text{desired target} = 1$$

Generator Loss

Since $y = 1$:

$$\begin{aligned} L_G &= -\left[1 \cdot \log \hat{y} + (1-1)\log(1-\hat{y})\right] \\ &= -\log \hat{y} \end{aligned}$$

Since $\hat{y} = D_\theta(G_\phi(z))$:

$$\boxed{L_G = -\log D_\theta(G_\phi(z))}$$

Generator Loss: Why $y = 1$, Not the True Label?

The image $\hat{x} = G_\phi(z)$ is actually fake — so shouldn't $y = 0$?

- The **discriminator's** loss uses the *true* label: $y = 1$ for real, $y = 0$ for fake
- The **generator's** loss asks a different question entirely

Two different questions:

- Discriminator's loss asks: *"Did you classify it correctly?"*
- Generator's loss asks: *"Did you fool the discriminator?"*

So for the generator, $y = 1$ is not the true label — it is the target the generator wants D_θ to output.

Generator Loss: Forward Pass

Forward pass:

$$z \rightarrow G_\phi(z) \rightarrow D_\theta \rightarrow \hat{y} = D_\theta\big(G_\phi(z)\big) \rightarrow L_G$$

Suppose:

$$\hat{y} = 0.2$$

Then:

$$L_G = -\log(0.2) \approx 1.609$$

The generator loss is **high** because the discriminator confidently identifies the generated image as fake.

Backward Pass: Generator

The gradient flows in the reverse direction:

$$L_G \rightarrow \hat{y} \rightarrow \hat{x} \rightarrow \phi \qquad \text{equivalently} \qquad L_G \rightarrow D_\theta \rightarrow G_\phi \rightarrow \phi$$

Using the chain rule:

$$\frac{\partial L_G}{\partial \phi} = \frac{\partial L_G}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \hat{x}} \cdot \frac{\partial \hat{x}}{\partial \phi}$$

Backward Pass: Generator

Since:

$$\hat{y} = D_\theta\big(G_\phi(z)\big), \qquad \hat{x} = G_\phi(z)$$

Substituting the definitions:

$$\frac{\partial L_G}{\partial \phi} = \frac{\partial L_G}{\partial D_\theta(G_\phi(z))} \cdot \frac{\partial D_\theta(G_\phi(z))}{\partial G_\phi(z)} \cdot \frac{\partial G_\phi(z)}{\partial \phi}$$

Backward Pass: Generator

Then the generator parameters are updated using gradient descent:

$$\phi \leftarrow \phi - \eta \frac{\partial L_G}{\partial \phi}$$

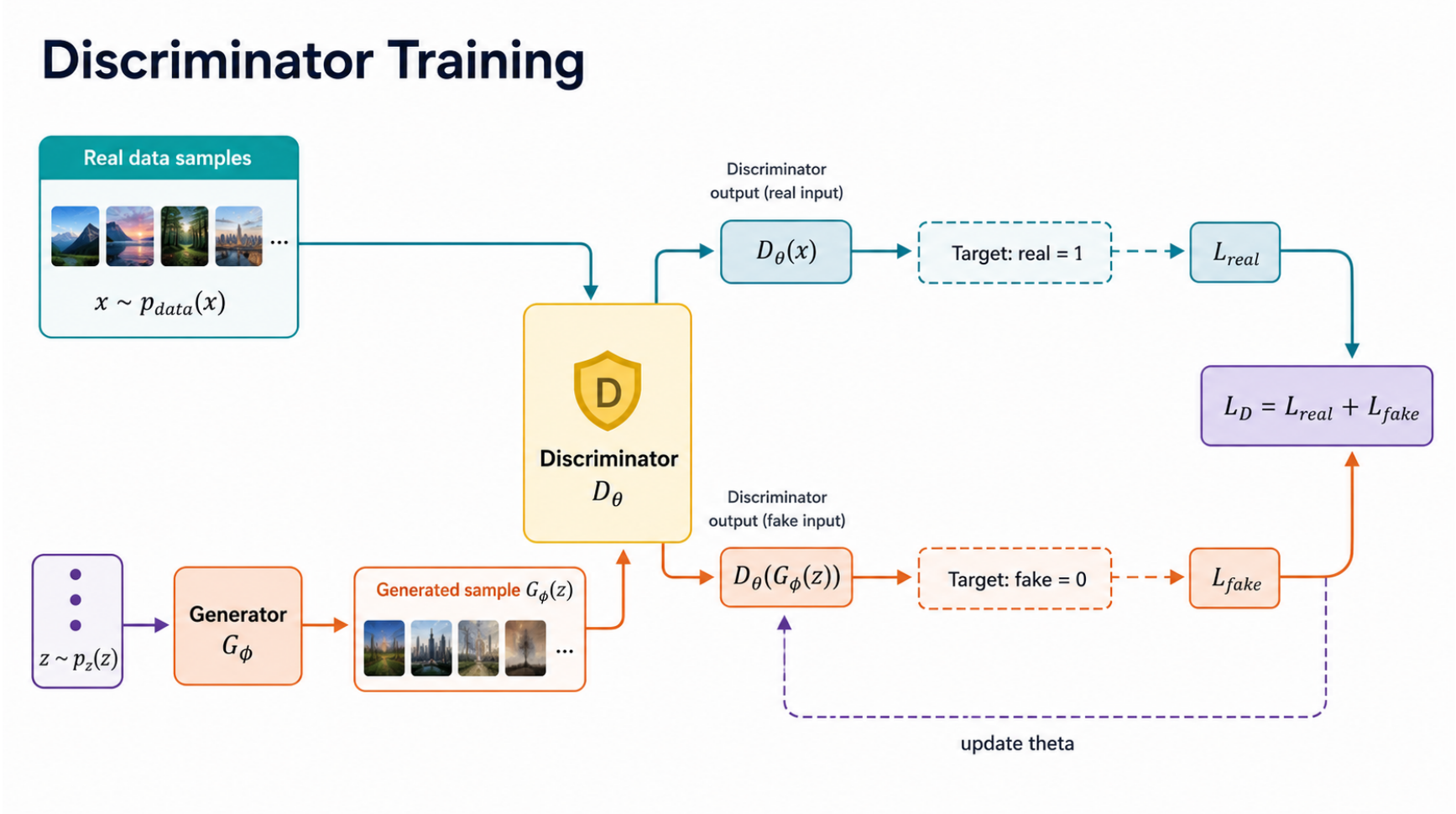
where η is the learning rate.

The purpose of the update:

Modify the generator so that the discriminator assigns a higher probability of being real to future generated images:

$$D_\theta\big(G_\phi(z)\big) \rightarrow 1$$

Discriminator Training



- Real samples $x \sim p_{data}(x)$ get target $= 1$; generated samples $G_\phi(z)$ get target $= 0$
- $L_D = L_{real} + L_{fake}$

Discriminator Loss: Classwork

The discriminator uses the *true* label for each sample:

- Real image x: true label $y = 1 \Rightarrow L_{real} = -\log D_\theta(x)$
- Fake image $\hat{x}$: true label $y = 0 \Rightarrow L_{fake} = -\log\left(1 - D_\theta(\hat{x})\right)$

$$L_D = L_{real} + L_{fake}$$

Given (from earlier steps):

$$w_d = [0.1, 0.1, 0.1, 0.1], \quad b_d = -1, \quad x = \begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}, \quad \hat{x} = \begin{bmatrix} 0.50 & -0.30 \\ 0.70 & 0.30 \end{bmatrix}$$

Question: Compute $\hat{y}_{real} = D_\theta(x)$, $\hat{y}_{fake} = D_\theta(\hat{x})$, then L_D.

Discriminator Loss: Worked Solution

Step 1 — Forward pass on the real image x:

$$\text{logit}_{real} = 0.1(2 + 4 + 6 + 8) - 1 = 1.0$$
$$\hat{y}_{real} = D_\theta(x) = \sigma(1.0) \approx 0.731$$

Step 2 — Forward pass on the fake image $\hat{x}$:

$$\text{logit}_{fake} = 0.1(0.50 - 0.30 + 0.70 + 0.30) - 1 = -0.88$$
$$\hat{y}_{fake} = D_\theta(\hat{x}) = \sigma(-0.88) \approx 0.293$$

Step 3 — Loss:

$$L_{real} = -\log(0.731) \approx 0.313, \qquad L_{fake} = -\log(1 - 0.293) \approx 0.347$$
$$L_D = L_{real} + L_{fake} \approx 0.660$$

Discriminator Loss

Consider BCE Loss:

$$L_{BCE} = -\left[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\right]$$

During discriminator training, there are two cases:

- Real image: $y = y_{real} = 1$
- Generated image: $y = y_{fake} = 0$

Discriminator Loss: 1. Loss for the Real Image

For a real image x:

$$\hat{y}_{real} = D_\theta(x)$$

Since $y_{real} = 1$, the BCE loss becomes:

$$L_{real} = -\left[1 \cdot \log \hat{y}_{real} + (1-1)\log(1-\hat{y}_{real})\right] = -\log \hat{y}_{real}$$

Therefore:

$$L_{real} = -\log D_\theta(x)$$

The discriminator wants: $D_\theta(x) \to 1$

Discriminator Loss: 2. Loss for the Generated Image

For a generated image $G_\phi(z)$:

$$\hat{y}_{fake} = D_\theta\big(G_\phi(z)\big)$$

Since $y_{fake} = 0$, the BCE loss becomes:

$$\begin{aligned} L_{fake} &= -\big[0 \cdot \log \hat{y}_{fake} + (1-0)\log(1-\hat{y}_{fake})\big] \\ &= -\log(1-\hat{y}_{fake}) \end{aligned}$$

Therefore:

$$L_{fake} = -\log\big(1 - D_\theta(G_\phi(z))\big)$$

The discriminator wants: $D_\theta(G_\phi(z)) \to 0$

Total Discriminator Loss

The real and fake losses are combined:

$$L_D = L_{real} + L_{fake}$$

Recall:

$$L_{real} = -\log D_\theta(x), \qquad L_{fake} = -\log\left(1 - D_\theta(G_\phi(z))\right)$$

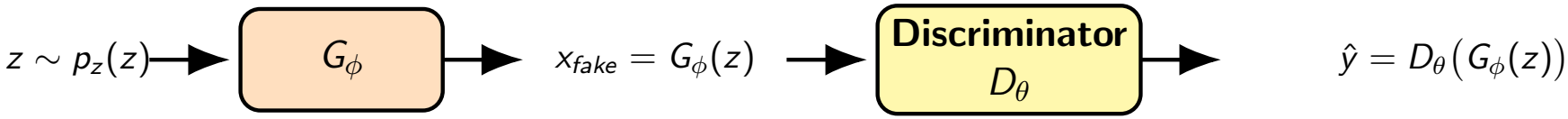
Therefore:

$$L_D = -\log D_\theta(x) - \log\left(1 - D_\theta(G_\phi(z))\right)$$

Equivalently:

$$L_D = -\Big[\log D_\theta(x) + \log\left(1 - D_\theta(G_\phi(z))\right)\Big]$$

Discriminator Loss for a Fake Sample: The Pipeline



Here:

- z is one random noise vector sampled from the noise distribution
- $x_{fake} = G_\phi(z)$ is the generator's output, passed to the discriminator
- $\hat{y} = D_\theta(G_\phi(z))$ is the discriminator's predicted probability that the generated sample is real

Discriminator Loss: An Optimization Objective

Neural networks are usually trained by minimizing a loss function. The discriminator parameters θ are updated so that this loss becomes as small as possible. Therefore, we write:

$$\min_{\theta} L_D(\theta; \phi) = \min_{\theta} -\Big[\log D_\theta(x) + \log\big(1 - D_\theta(G_\phi(z))\big)\Big]$$

Here, G_ϕ is kept fixed while the discriminator parameters θ are updated.

Converting Loss Minimization to Objective Maximization

A general identity:

$$\min_D \big[-A(D) \big] \iff \max_D A(D)$$

Minimizing the negative of a quantity is the same as maximizing the quantity itself.

Converting Loss Minimization to Objective Maximization

Since the discriminator loss is the negative of the objective, minimizing L_D is equivalent to maximizing V_D:

$$\min_{\theta} -\Big[\log D_\theta(x) + \log\big(1 - D_\theta(G_\phi(z))\big)\Big]$$

is equivalent to:

$$\max_{\theta} \Big[\log D_\theta(x) + \log\big(1 - D_\theta(G_\phi(z))\big)\Big]$$

GAN Unit: Progress Tracker (Theory)

Section	Covered	Status
Motivation	VAE limits, adversarial idea, G/D cartoons, competition loop	Done
Generator	Training pass, θ-frozen, forward pass, loss derivation, backward pass	Done
Discriminator	Training pass, forward pass, loss derivation, min/max framing, classwork	Done
Formal Objective	Expectation notation (real images)	Partial

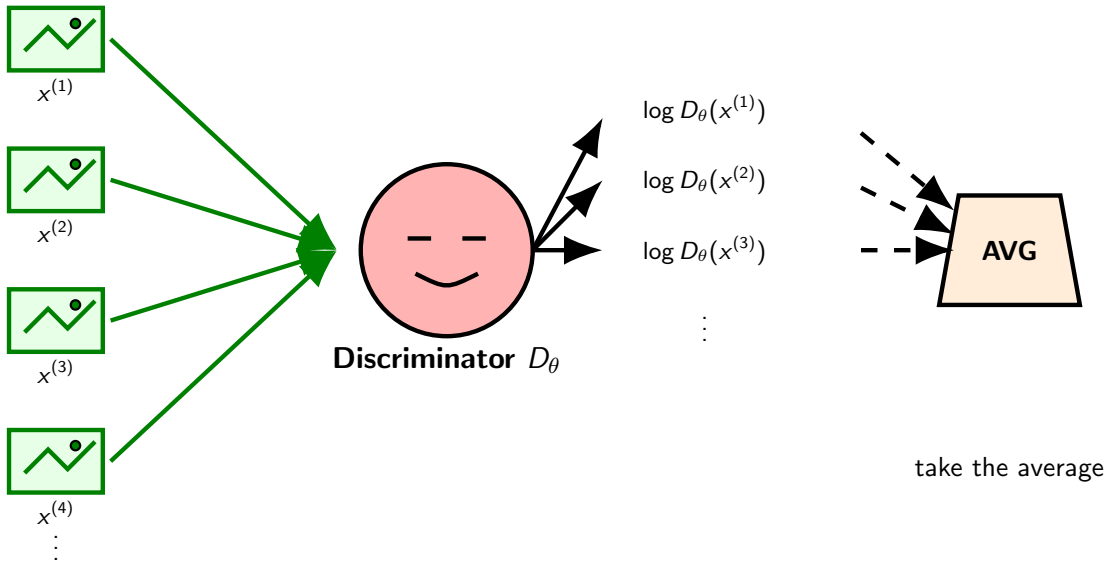
Remaining: fake-image expectation, full minimax $V(D, G)$, Discriminator backward pass (theory), training algorithm, failure modes, Nash equilibrium, VAE vs. GAN, applications

GAN Unit: Progress Tracker (Lab, Experiment 5)

Step	Content	Result
1	$z \in \mathbb{R}^2$, real image $x \in \mathbb{R}^{2\times 2}$	$z = [0.50, -0.30]$
2	Generator forward pass	$\hat{x} = [[0.50, -0.30], [0.70, 0.30]]$
3	Discriminator forward pass	$D_\theta(x) = 0.731$, $D_\theta(\hat{x}) = 0.293$
4	Generator loss L_G	$L_G \approx 1.227$
5–6	Generator backward pass + update ϕ	$L_G : 1.2270 \rightarrow 1.2243$
7	Discriminator loss L_D	$L_D \approx 0.660$
8–9	Discriminator backward pass + update θ	$L_D : 0.6602 \rightarrow 0.4085$
10	*Remaining: multi-iteration training loop*	

From a Single Sample to the Expectation

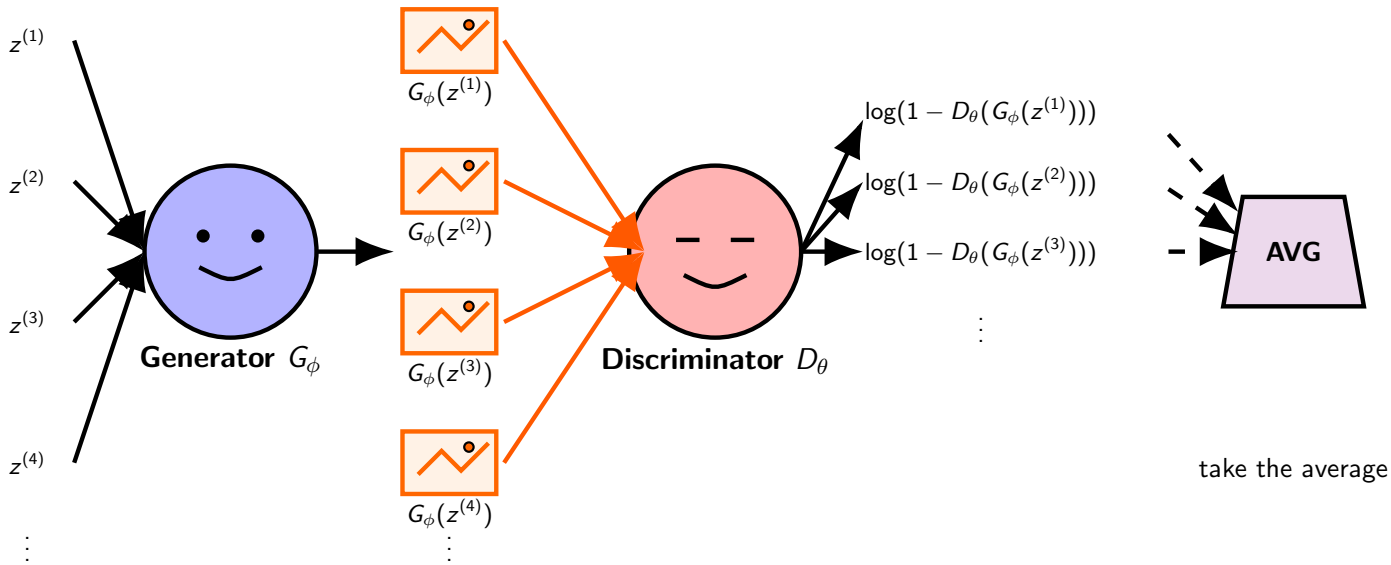
This expression uses one real image x and one noise sample z. During training, we use *many* samples — so we take their average:



$$\mathbb{E}_{x \sim p_{data}} \left[\log D_\theta(x) \right] \approx \frac{1}{m} \sum_{i=1}^{m} \log D_\theta \left(x^{(i)} \right)$$

Expectation for Generated Images

For generated images, we sample many noise vectors, generate fake samples, and average the discriminator's response:



$$\mathbb{E}_{z\sim p_z}\left[\log\left(1 - D_\theta(G_\phi(z))\right)\right] \approx \frac{1}{m}\sum_{i=1}^{m}\log\left(1 - D_\theta\left(G_\phi(z^{(i)})\right)\right)$$

Discriminator Objective Function

What is V_D? The discriminator's value function — a score built from both the real-image and generated-image expectations, which θ is optimized to maximize:

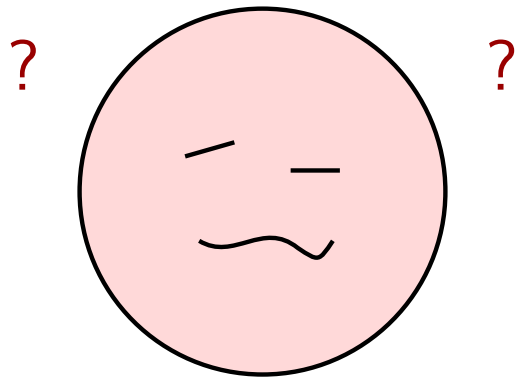
$$V_D(\theta;\phi) = \mathbb{E}_{x\sim p_{data}}\Big[\log D_\theta(x)\Big] + \mathbb{E}_{z\sim p_z}\Big[\log\big(1 - D_\theta(G_\phi(z))\big)\Big]$$

The semicolon in $V_D(\theta;\phi)$ indicates that θ is being optimized while ϕ is treated as fixed.

Then write:

$$\max_\theta V_D(\theta;\phi)$$

Discriminator: Untrained



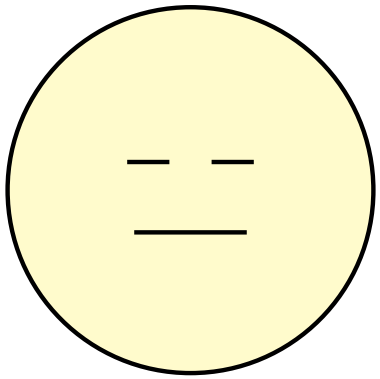
Initially, the discriminator may not identify the real image correctly. Suppose it gives:

$$D_\theta(x) = 0.1$$

$$\log D_\theta(x) = \log(0.1) \approx -2.303$$

This is a very low negative value — the discriminator's performance on this real sample is poor.

Discriminator: Partially Trained



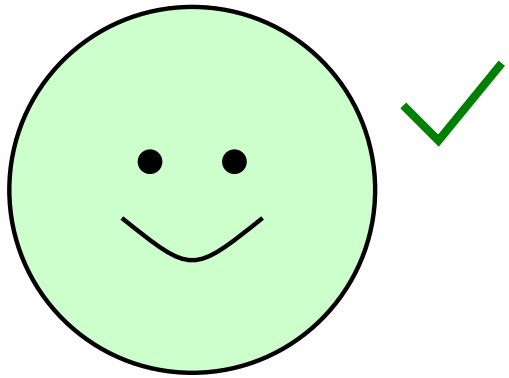
After some training, suppose:

$$D_\theta(x) = 0.5$$

$$\log(0.5) \approx -0.693$$

Better than before, but still far from confident.

Discriminator: Fully Trained



If the discriminator improves further:

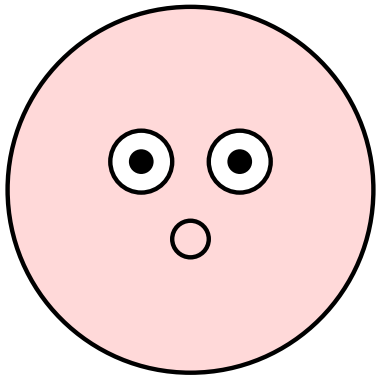
$$D_\theta(x) = 0.9 \Rightarrow \log(0.9) \approx -0.105$$

Finally, as $D_\theta(x) \to 1$:

$$\log D_\theta(x) \to \log(1) = 0 \quad \text{(the maximum possible value)}$$

Therefore: $D_\theta(x) \uparrow \Rightarrow \log D_\theta(x) \uparrow$. Maximizing the real-image term pushes $D_\theta(x) \to 1$.

Discriminator on Fakes: Untrained



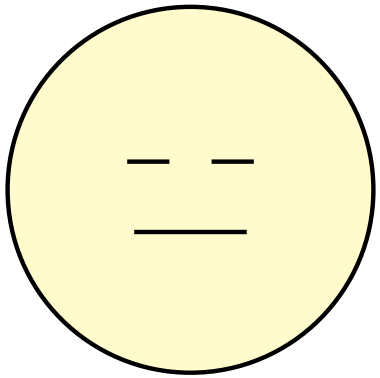
Initially, the discriminator may incorrectly classify. Suppose it gives:

$$D_\theta\big(G_\phi(z)\big) = 0.9$$

$$1 - D_\theta\big(G_\phi(z)\big) = 0.1 \;\Rightarrow\; \log(0.1) \approx -2.303$$

The discriminator was fooled — performance on this generated sample is poor.

Discriminator on Fakes: Partially Trained



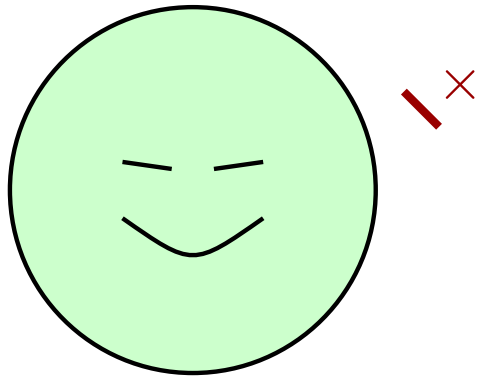
After some training, suppose the discriminator improves and gives:

$$D_\theta\big(G_\phi(z)\big) = 0.5$$

$$1 - D_\theta\big(G_\phi(z)\big) = 0.5 \;\Rightarrow\; \log(0.5) \approx -0.693$$

Less certain than before — the discriminator is starting to notice something is off.

Discriminator on Fakes: Fully Trained



If the discriminator improves further, suppose:

$$D_\theta\big(G_\phi(z)\big) = 0.1 \;\Rightarrow\; 1 - D_\theta\big(G_\phi(z)\big) = 0.9 \;\Rightarrow\; \log(0.9) \approx -0.105$$

Finally, as $D_\theta(G_\phi(z)) \to 0$:

$$1 - D_\theta\big(G_\phi(z)\big) \to 1 \;\Rightarrow\; \log\big(1 - D_\theta\big(G_\phi(z)\big)\big) \to \log(1) = 0$$

The term is largest when $D_\theta(G_\phi(z))$ becomes close to zero — the discriminator confidently spots the fake.

Generator Objective Function

The GAN value function is:

$$V_G(\phi;\theta) = \mathbb{E}_{x\sim p_{data}}\left[\log D_\theta(x)\right] + \mathbb{E}_{z\sim p_z}\left[\log\left(1 - D_\theta(G_\phi(z))\right)\right]$$

The first term contains only real samples and the discriminator:

$$\mathbb{E}_{x\sim p_{data}}\left[\log D_\theta(x)\right]$$

It does not depend on the generator parameters ϕ. Therefore, the generator cannot change this term.

Generator Objective Function

The second term contains $G_\phi(z)$:

$$\mathbb{E}_{z\sim p_z}\Big[\log\big(1 - D_\theta(G_\phi(z))\big)\Big]$$

Therefore, during the generator update, we optimize only this term.

The generator wants the discriminator to classify generated samples as real. Therefore:

$$D_\theta\big(G_\phi(z)\big) \to 1$$

Then:

$$1 - D_\theta\big(G_\phi(z)\big) \to 0$$

Generator Objective Function

$D_\theta(G_\phi(z))$	$1 - D_\theta(G_\phi(z))$	$\log(1 - D_\theta(G_\phi(z)))$
0.1	0.9	-0.105
0.5	0.5	-0.693
0.9	0.1	-2.303
0.99	0.01	-4.605
0.999	0.001	-6.908

Therefore:

$$\log\left(1 - D_\theta(G_\phi(z))\right) \to -\infty$$

So the generator tries to make this term as small as possible. Hence:

$$\min_\phi \ \mathbb{E}_{z\sim p_z}\left[\log\left(1 - D_\theta(G_\phi(z))\right)\right]$$

GAN Objective Function

The discriminator maximizes with respect to θ:

$$\max_{\theta} V(\theta; \phi)$$

During this update, the generator parameters ϕ are kept fixed.

The generator minimizes with respect to ϕ:

$$\min_{\phi} V(\theta; \phi)$$

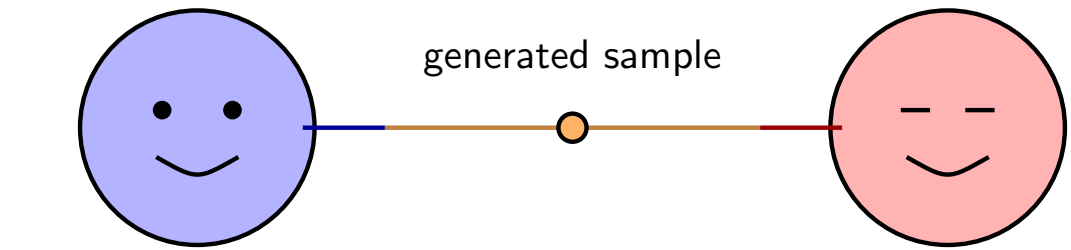
During this update, the discriminator parameters θ are kept fixed.

Combining both gives the original GAN minimax objective:

$$\min_{\phi} \max_{\theta} V(\theta; \phi)$$

GAN Objective Function: The Minimax Game

$$\min_G \max_D \; \mathbb{E}_{x \sim p_{data}} \Big[\log D_\theta(x) \Big] + \mathbb{E}_{z \sim p_z} \Big[\log \big(1 - D_\theta(G_\phi(z)) \big) \Big]$$



Generator G_ϕ
wants $D_\theta(G_\phi(z)) \to 1$

Discriminator D_θ
wants $D_\theta(G_\phi(z)) \to 0$

Their goals are opposite: the discriminator maximizes V, the generator minimizes it.

GAN Training Algorithm

For each training iteration, repeat:

Step 1: Update the Discriminator (k steps)

For k steps:

- Sample a minibatch of m noise samples $\{z^{(1)}, \ldots, z^{(m)}\}$ from $p_z(z)$
- Sample a minibatch of m real examples $\{x^{(1)}, \ldots, x^{(m)}\}$ from $p_{data}(x)$
- Update θ by **ascending** its stochastic gradient:

$$\nabla_\theta \frac{1}{m} \sum_{i=1}^{m} \left[\log D_\theta(x^{(i)}) + \log \left(1 - D_\theta(G_\phi(z^{(i)}))\right) \right]$$

GAN Training Algorithm

Step 2: Update the Generator (1 step)

- Sample a *new* minibatch of m noise samples $\{z^{(1)}, \ldots, z^{(m)}\}$ from $p_z(z)$
- Update ϕ by **descending** its stochastic gradient:

$$\nabla_\phi \frac{1}{m} \sum_{i=1}^{m} \log\left(1 - D_\theta(G_\phi(z^{(i)}))\right)$$

Repeat Steps 1–2 for the desired number of training iterations.

k is a hyperparameter — some implementations use $k = 1$ (alternate every step), others use larger k to let the discriminator stay ahead of the generator.

GAN Convergence

The real training samples are drawn from the real-data distribution:

$$x \sim p_{data}(x)$$

The generated samples are produced as:

$$z \sim p_z(z), \qquad x_{fake} = G_\phi(z)$$

The collection of generated samples follows the generator distribution:

$$x_{fake} \sim p_g(x)$$

Here, $p_{data}(x)$ is the real-data distribution, and $p_g(x)$ is the distribution of samples generated by G_ϕ.

GAN Convergence: As Training Progresses

As training progresses, the discriminator provides gradients that are backpropagated through D_θ to the generator.

The generator updates its parameters ϕ so that:

$$G_\phi(z)$$

produces more realistic samples.

GAN Convergence: As Training Progresses

As the generated samples improve, the generated distribution moves closer to the real-data distribution:

$$p_g(x) \rightarrow p_{data}(x)$$

At the sample level, this means generated samples become harder to distinguish from real samples.

GAN Convergence: As Training Progresses

As the generator improves, $D_\theta(G_\phi(z))$ begins to increase.

For example:

$$D_\theta\big(G_\phi(z)\big) = 0.1 \;\text{ may later become }\; 0.4 \;\text{ and then }\; 0.5$$

This means the generated images are becoming more difficult for the discriminator to classify as fake.

GAN Convergence: At Ideal Convergence

At the ideal GAN equilibrium:

$$p_g(x) = p_{data}(x)$$

The discriminator can no longer distinguish whether a sample came from the real-data distribution or the generator distribution.

Therefore:

$$D_\theta(x) = \frac{1}{2}$$

for samples from either distribution.

In particular:

$$D_\theta(x) \approx 0.5 \qquad \text{and} \qquad D_\theta\big(G_\phi(z)\big) \approx 0.5$$

GAN Convergence: What This Means

This does not mean that the discriminator is poorly trained. It means that the generator has learned to produce samples whose distribution matches the real-data distribution.

GAN Convergence:

GAN convergence refers to the ideal state in which the generated-data distribution matches the real-data distribution, and the discriminator can no longer distinguish real samples from generated samples.

Nash Equilibrium in GANs

In GANs, the two players are:

Generator G and Discriminator D

At equilibrium:

$$p_g(x) = p_{data}(x)$$

and:

$$D(x) = 0.5$$

Nash Equilibrium in GANs

Why neither player can improve:

- The discriminator cannot improve because real and generated samples follow the same distribution.
- The generator cannot improve further because it has already matched the real-data distribution.

Nash equilibrium is a state in which neither the generator nor the discriminator can obtain a better outcome by changing its own parameters alone, while the parameters of the other network are kept unchanged.

What is Random Noise?

Latent Space:

Latent space refers to an abstract, often lower-dimensional representation of data that captures its essential features and underlying patterns.

Random noise is a vector in the latent space. It's a latent vector Z.

$$Z = \begin{bmatrix} 0.6 \\ 0.5 \\ 0.1 \\ \vdots \\ 0.7 \end{bmatrix}$$

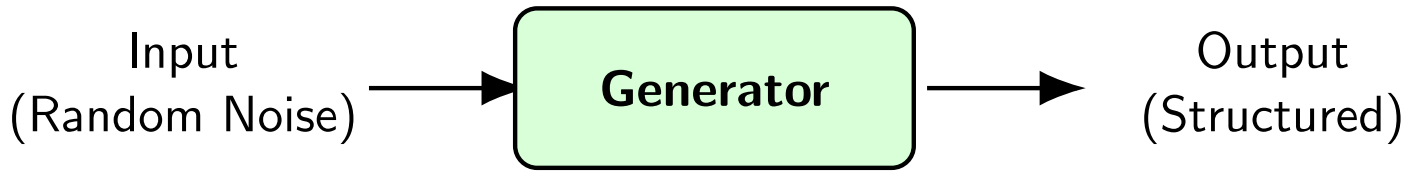
$Z \sim \mathcal{N}(0, 1)$ — Standard Normal (Gaussian) noise

Each component of Z is drawn independently from a normal distribution with:

- Mean $= 0$
- Variance $= 1$

What is Random Noise?

If it is 100 dimensions, then it has 100 random numbers.



But no control on what gets generated.

Conditional GANs

Conditional GANs enable controlled and guided data generation.

Reference: Conditional Generative Adversarial Nets

arXiv:1411.1784v1 [cs.LG] 6 Nov 2014

Conditional Generative Adversarial Nets

Mehdi Mirza
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7
mirzamom@iro.umontreal.ca

Simon Osindero
Flickr / Yahoo Inc.
San Francisco, CA 94103
osindero@yahoo-inc.com

Abstract

Generative Adversarial Nets [8] were recently introduced as a novel way to train generative models. In this work we introduce the conditional version of generative adversarial nets, which can be constructed by simply feeding the data, y, we wish to condition on to both the generator and discriminator. We show that this model can generate MNIST digits conditioned on class labels. We also illustrate how this model could be used to learn a multi-modal model, and provide preliminary examples of an application to image tagging in which we demonstrate how this approach can generate descriptive tags which are not part of training labels.

1 Introduction

Generative adversarial nets were recently introduced as an alternative framework for training generative models in order to sidestep the difficulty of approximating many intractable probabilistic computations.

Adversarial nets have the advantages that Markov chains are never needed, only backpropagation is used to obtain gradients, no inference is required during learning, and a wide variety of factors and interactions can easily be incorporated into the model.

Furthermore, as demonstrated in [8], it can produce state of the art log-likelihood estimates and realistic samples.

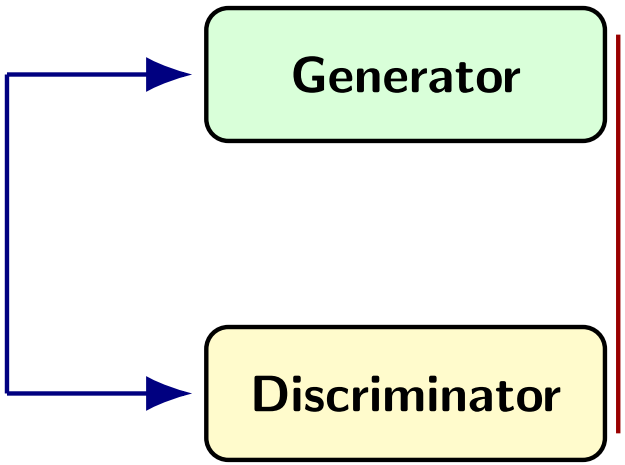
In an unconditioned generative model, there is no control on modes of the data being generated. However, by conditioning the model on additional information it is possible to direct the data generation process. Such conditioning could be based on class labels, on some part of data for inpainting like [5], or even on data from different modality.

In this work we show how can we construct the conditional adversarial net. And for empirical results we demonstrate two set of experiment. One on MNIST digit data set conditioned on class labels and one on MIR Flickr 25,000 dataset [10] for multi-modal learning.

Mirza, M., & Osindero, S. (2014). *Conditional Generative Adversarial Nets*.

What is Conditional GAN

Conditioning your GAN using additional information



Conditioned on the class labels

(0–9) digits for MNIST dataset

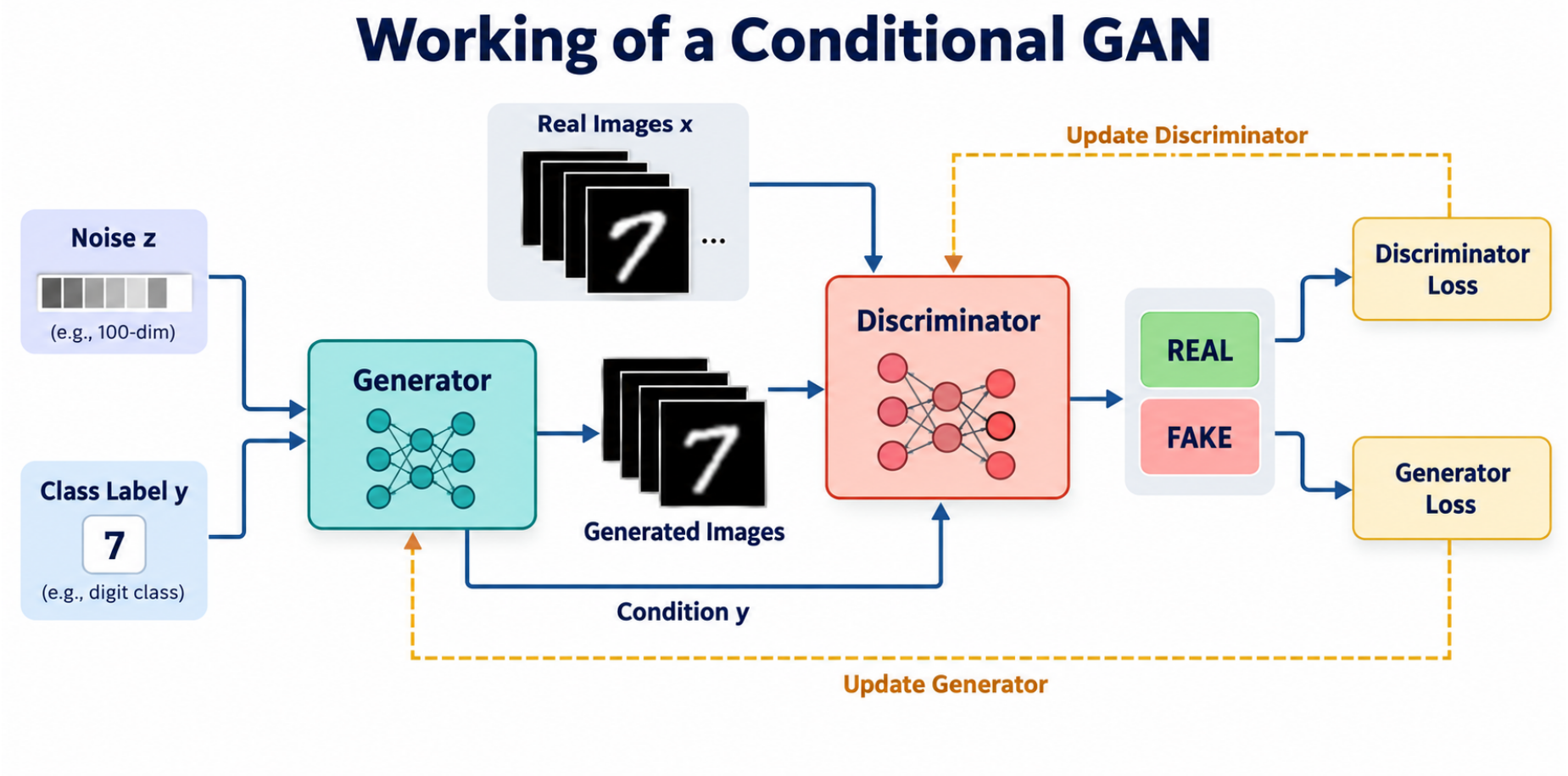
What is Conditional GAN

When we are training our Generator and Discriminator, we will provide class labels along with the images.

GANs can also be conditioned on:

- ✓ Other modalities (image to image translation)
- ✓ Text prompts

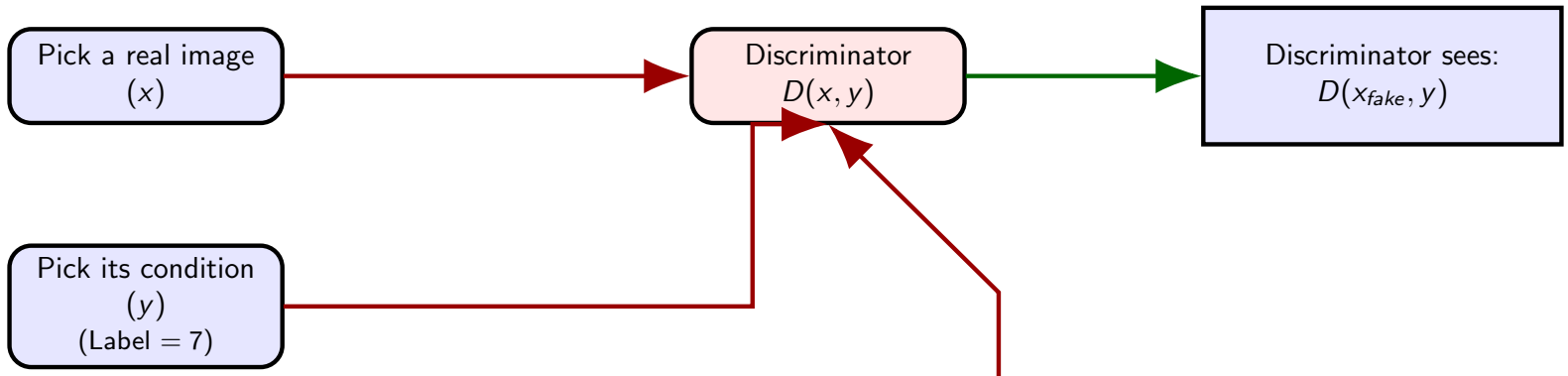
Working of a Conditional GAN



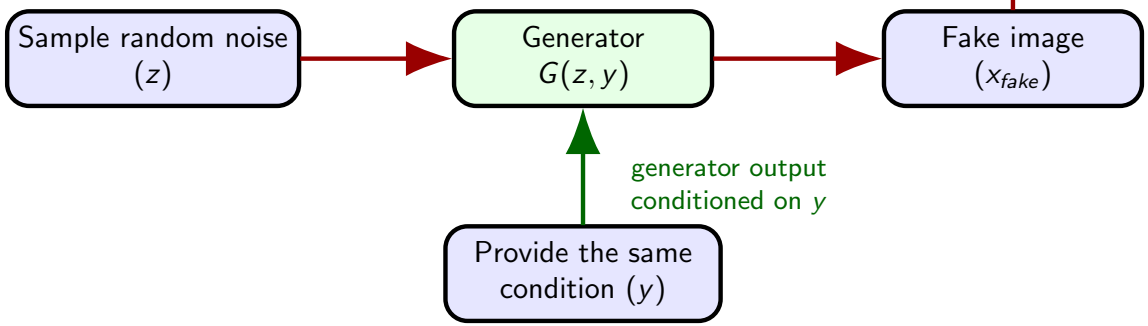
- Both Generator and Discriminator receive the class label y alongside their usual input (z for the Generator, the image for the Discriminator)
- Generator: $G(z, y) \rightarrow$ generated image conditioned on y

Training Loop in Conditional GAN

Step 1:



Step 2:



Training Loop in Conditional GAN

Step 3:

Train Discriminator

$$D(x, y) \rightarrow 1 \ (real)$$

$$D(G(z, y), y) \rightarrow 0 \ (fake)$$

The discriminator learns two things:

- Does the image match the condition?
- Is it real?

Training Loop in Conditional GAN

Step 4:

Train Generator

$$D\big(G(z, y), y\big) \to 1$$

The generator learns two things:

- Given label y, generate an image that looks real, **AND**
- Matches y

Training Loop in Conditional GAN

Step 5:

Repeat

- ✓ Generator improves image quality
- ✓ Discriminator improves detection
- ✓ Eventually generator learns to produce realistic, label-consistent samples

Discriminator Loss in cGAN

Goal of Discriminator:

- ✓ Predict real pairs as real
- ✓ Predict fake pairs as fake

Discriminator Loss in cGAN: Real Loss

measures how well D correctly classifies real data as real

$$\mathcal{L}_D^{real} = -\mathbb{E}_{x \sim p_{data}(x)} \big[\log D(x, y) \big]$$

Discriminator Loss in cGAN: What Each Term Means

$\mathbb{E}_{x \sim p_{data}(x)}$ — expectation (average) over the real dataset

Here, $p_{data}(x)$ is the true data distribution.

$D(x, y) = P(\text{real} \mid x, y)$

The output is a probability score between 0 and 1.

Discriminator Loss in cGAN: Why the Negative Sign

Discriminator wants:

$$D(x, y) \to 1$$

The negative sign is given for minimizing the loss — since $\log D(x, y)$ increases as $D(x, y) \to 1$, minimizing $-\log D(x, y)$ pushes $D(x, y)$ toward 1.

Discriminator Loss in cGAN: Worked Example

If $D(x, y) = 0.9$ (confident, correct):

$$\log D(x, y) = \log(0.9) \approx -0.105$$

If $D(x, y) = 0.2$ (unconfident, poor performance on this real pair):

$$\log D(x, y) = \log(0.2) \approx -1.609$$

Higher $D(x, y)$ gives a less negative $\log D(x, y)$ — and therefore a lower loss $-\log D(x, y)$, exactly matching the discriminator's goal.

Discriminator Loss in cGAN: Fake Loss

Measures how well the *discriminator* can detect generated data as fake

$$\mathcal{L}_D^{fake} = -\mathbb{E}_{z\sim p_z(z)}\Big[\log\big(1 - D(G(z,y),y)\big)\Big]$$

Discriminator Loss in cGAN: What Each Term Means

$\mathbb{E}_{z\sim p_z(z)}$ — expectation (average) over noise samples

Discriminator wants:

$$D\big(G(z,y),y\big) \to 0$$

Discriminator Loss in cGAN: Total Loss

Total Discriminator Loss:

$$\mathcal{L}_D = -\mathbb{E}_{x\sim p_{data}(x)}\Big[\log D(x,y)\Big] \; - \; \mathbb{E}_{z\sim p_z(z)}\Big[\log\big(1 - D(G(z,y),y)\big)\Big]$$

The total loss combines the real-loss term (Real Loss) and the fake-loss term (Fake Loss).

Discriminator Loss in cGAN: Key Terms Explained

$G(z, y)$:

Produces a fake sample conditioned on y.

$D\big(G(z, y), y\big) = P(\text{real} \mid \text{generated sample}, y)$:

The discriminator's estimated probability that this generated sample, given condition y, is real.

Generator Loss in cGAN

Goal of Generator

Generator tries to fool the discriminator by producing samples that look real for that specific condition.

Generator Loss in cGAN: Standard Generator Loss

Standard Generator Loss

$$\mathcal{L}_G = \mathbb{E}_{z,y}\Big[\log\big(1 - D(G(z,y),y)\big)\Big]$$

But the vanishing gradient problem occurs if this generator loss is taken.

Generator Loss in cGAN: Practical Non-Saturating Loss

$$\mathcal{L}_G = -\mathbb{E}_{z\sim p(z)}\Big[\log D\big(G(z,y),y\big)\Big]$$

Generator Loss in cGAN: What Each Term Means

$\mathbb{E}_{z\sim p(z)}$ — expectation (average) over noise samples

Measures how well the generator can fool the discriminator

Negative term is added to minimize the loss.

Generator Loss in cGAN: What the Generator Wants

Generator wants to maximize this value:

$$D\big(G(z, y), y\big) \to 1$$

$D(G(z, y), y)$ is the **probability score** that the Discriminator assigns to the Generator's fake creation.

Objective Function in cGAN

In cGAN, the overall objective is a min–max game between Generator G and Discriminator D, both conditioned on y:

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{data}(x)}\big[\log D(x,y)\big] + \mathbb{E}_{z\sim p_z(z)}\Big[\log\big(1 - D(G(z,y))\big)\Big]$$

Objective Function in cGAN: Roles

$\min_G \max_D$

Discriminator tries to **maximize** its ability to distinguish real from fake, while Generator tries to **minimize** the Discriminator's success.

$V(D, G)$

Overall objective function that both networks are competing to optimize.

Objective Function in cGAN: The Two Terms

$\mathbb{E}_{x \sim p_{data}(x)}[\log D(x, y)]$

Encourages the discriminator to classify real samples correctly.

$\mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z, y)))]$

Pushes the generator to produce samples that the discriminator classifies as real.

Objective Function in cGAN: Outcome

- ✓ As training progresses, both the generator and discriminator improve.
- ✓ This adversarial process results in the generator producing more realistic data conditioned on the input information.

Key Difference between Vanilla GAN and Conditional GAN

Vanilla GAN	**Conditional GAN**
Generator's output depends only on the random noise, i.e. $G(z)$	Generator's output depends on both random noise and the class label, i.e. $G(z, y)$
$D(x)$: Probability that input x is **real** (from data)	$D(x, y)$: Probability that input x is **real** (from data) and matches condition y
No control on generation	Controlled generation

Where Conditional GANs Are Actually Used

- **Class-controlled data augmentation:** generating more samples of a specific, underrepresented class — e.g. rare disease categories in medical imaging datasets, where real examples are scarce
- **Attribute-controlled face generation:** generating faces with specified attributes (age, gender, expression, hairstyle) — used in research tools and creative/design applications
- **Fashion and product design:** generating clothing or product images conditioned on style, category, or color attributes for design prototyping
- **Image-to-image translation (foundation for):** cGAN's conditioning idea underlies later architectures like Pix2Pix, which map an input image (e.g. a sketch or segmentation map) to a realistic output image
- **Text-to-image generation (early approaches):** architectures such as StackGAN and AttnGAN condition the generator on text embeddings, an early step toward today's text-to-image systems

Applications of Conditional GAN

Image to Image Translation

- Sketch $\rightarrow$ original image
- Day time $\rightarrow$ night time

Super-Resolution

Generate high-resolution images from low-resolution inputs.

Style Transfer

Convert photo $\rightarrow$ painting style (e.g. Van Gogh style).

Text to Image Translation

E.g. "Create a red bird with blue wings."

Where These Are Actually Used

- **Sketch/Day-Night translation (Pix2Pix-style):** generating synthetic day/night or adverse-weather driving scenes to augment training data for autonomous vehicle perception systems; concept-art and product-design tools that turn a rough sketch into a rendered image
- **Super-resolution:** enhancing satellite and aerial imagery, restoring old or low-quality photos and video footage, and improving low-resolution medical scans for clearer diagnosis
- **Style transfer:** consumer apps such as Prisma and DeepArt, and artistic filters built into photo-editing software
- **Text-to-image:** the conditioning idea here is a direct ancestor of today's text-to-image systems (DALL-E, Midjourney, Stable Diffusion) — though most current production systems use diffusion models rather than GANs, the core idea of conditioning generation on a text description originates from work in this space

Reference: Image-to-Image Translation with Conditional Adversarial Networks

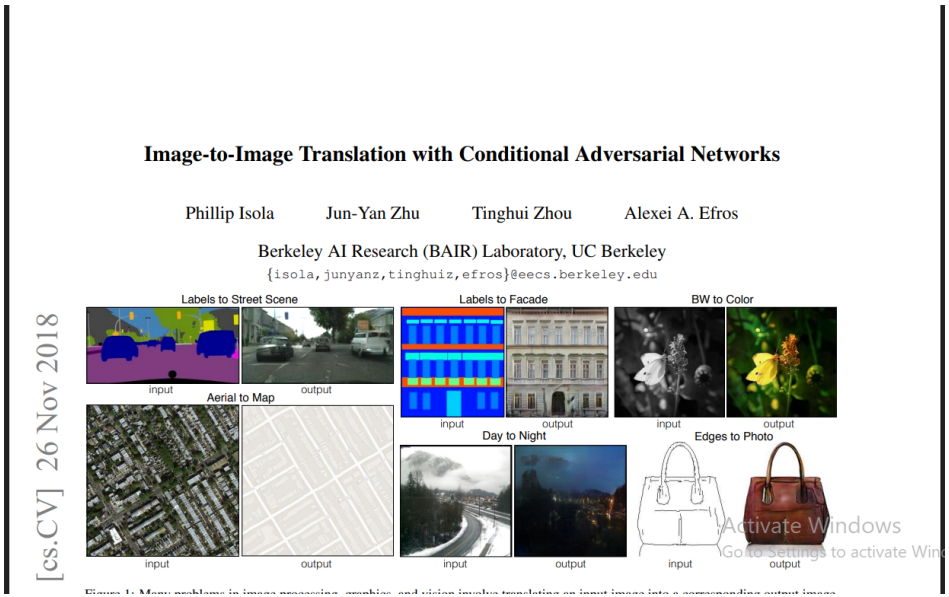


Image-to-Image Translation with Conditional Adversarial Networks

Phillip Isola Jun-Yan Zhu Tinghui Zhou Alexei A. Efros

Berkeley AI Research (BAIR) Laboratory, UC Berkeley

{isola,junyanz,tinghuiz,efros}@eecs.berkeley.edu

[cs.CV] 26 Nov 2018

Activate Windows
Go to Settings to activate Win

Figure:

Isola, P., Zhu, J.-Y., Zhou, T., & Efros, A. A. (2017). *Image-to-Image Translation with Conditional Adversarial Networks*. CVPR. (Berkeley AI Research)

Pix2Pix and cGAN

- ✓ cGAN is a generic conditional generation framework
  - Can use labels, text, images, etc. as condition
- ✓ Pix2Pix is itself a **Conditional GAN (cGAN)** specifically designed for **paired image-to-image translation**

Pix2Pix: Different Image Translation Tasks

The model learns paired input-to-output transformations: sketch $\rightarrow$ painting, low-resolution $\rightarrow$ super-resolution, corrupted $\rightarrow$ clean.

Core idea: preserve structure, change or improve appearance.

Pix2Pix: Paired Image-to-Image Translation

pix2pix: Paired Image-to-Image Translation

Learn a mapping from input image domain X to output image domain Y

Input: Edge Sketch



Generator learns translation

Output: Realistic Photo



Input: Map



Generator learns translation

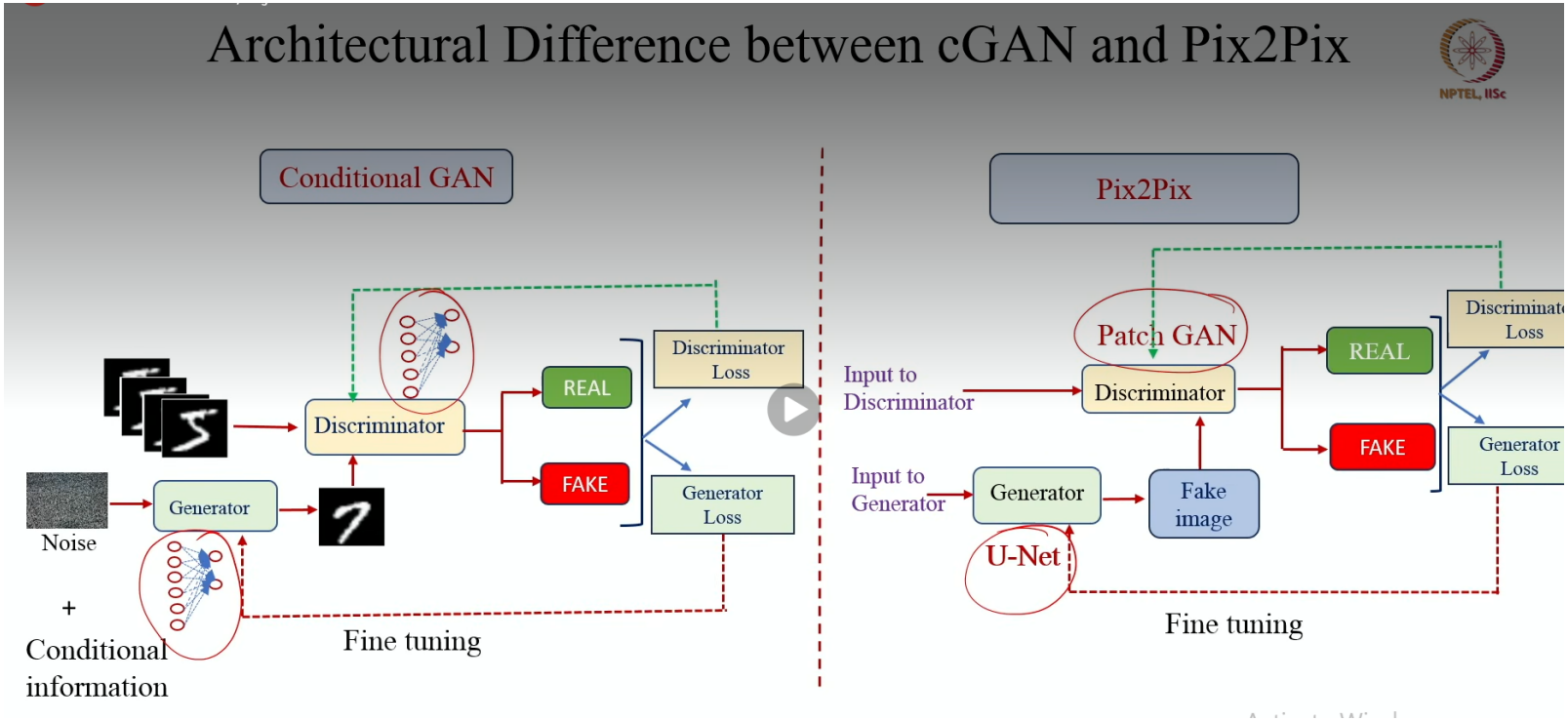
Output: Satellite View



Training pairs: (x, y)

Goal: preserve layout, change appearance.

Architectural Difference between cGAN and Pix2Pix



Pix2Pix's discriminator uses a **PatchGAN** architecture (classifies overlapping patches, not the whole image at once)

Pix2Pix's generator uses a **U-Net** architecture (encoder-decoder with skip connections, preserving fine spatial detail)